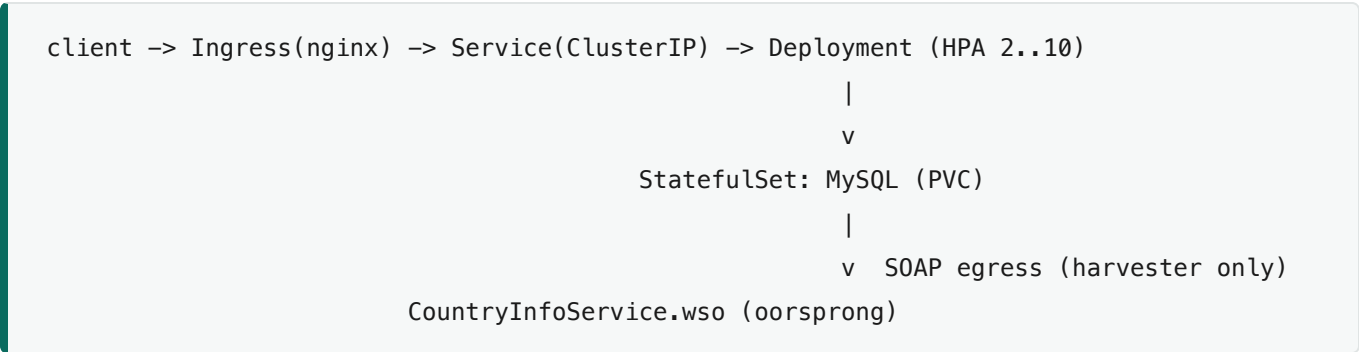


Kubernetes Deployment Guide

Deploys RDAS (Spring Boot + MySQL) using the manifests under `k8s/` . All resources live in the `rdas` namespace.

1. Components

Component	Kind	File
Namespace	Namespace	namespace.yaml
App config	ConfigMap	configmap.yaml
Credentials	Secret	secret.yaml
Database	StatefulSet + headless Service + PVC	mysql-deployment.yaml
Application	Deployment (2 replicas)	deployment.yaml
App network	Service (ClusterIP, 80 -> 8080)	service.yaml
Ingress	Ingress (nginx)	ingress.yaml
Autoscaling	HorizontalPodAutoscaler (2->10 @70% CPU)	hpa.yaml



2. Prerequisites

- A Kubernetes cluster and `kubectl` (`kubectl cluster-info`).
- An ingress controller (ingress-nginx) if you use `ingress.yaml` .
- `metrics-server` if you want the HPA to scale on CPU (`kubectl top pods`).
- Cluster outbound internet (the harvester calls the public SOAP service).
- The image available to the cluster (built or loaded - see below).

3. Build and publish the image

The manifest references `reference-data-aggregation-service:1.0.0`.

```
# minikube: build into the cluster daemon
eval $(minikube docker-env)
docker build -t reference-data-aggregation-service:1.0.0 .

# kind: build then load
docker build -t reference-data-aggregation-service:1.0.0 .
kind load docker-image reference-data-aggregation-service:1.0.0

# Docker Desktop Kubernetes: a local build is visible to the cluster directly.
docker build -t reference-data-aggregation-service:1.0.0 .

# real registry
docker build -t <registry>/reference-data-aggregation-service:1.0.0 .
docker push <registry>/reference-data-aggregation-service:1.0.0
# then update image: in k8s/deployment.yaml
```

4. Deploy step by step

1. Namespace

```
kubectl apply -f k8s/namespace.yaml
```

2. Config + secrets

```
kubectl apply -f k8s/configmap.yaml -f k8s/secret.yaml
```

3. Database (skip if using an external DB – see section 6)

```
kubectl apply -f k8s/mysql-deployment.yaml
```

```
kubectl -n rdas rollout status statefulset/mysql --timeout=180s
```

```
kubectl -n rdas get pvc # mysql-data-mysql-0 should be Bound
```

4. Application

```
kubectl apply -f k8s/deployment.yaml -f k8s/service.yaml
```

```
kubectl -n rdas rollout status deployment/rdas-app --timeout=180s
```

```
kubectl -n rdas get endpoints rdas-app # should list 2 pod IPs
```

5. Autoscaling + ingress

```
kubectl apply -f k8s/hpa.yaml -f k8s/ingress.yaml
```

```
kubectl -n rdas get hpa
```

5. Verify

Port-forward the Service (no ingress needed)

```
kubectl -n rdas port-forward svc/rdas-app 8080:80
```

In another terminal:

```
curl http://localhost:8080/actuator/health
```

```
curl http://localhost:8080/api/v1/admin/catalog/status
```

```
curl "http://localhost:8080/api/v1/countries?continent=Africa&size=5"
```

The first startup triggers a background harvest; the catalog fills in 2-3 minutes. Until then `countryCount` climbs and the overall health is `DOWN` (catalog empty) while `readiness` stays `UP` - by design, so the pod serves as soon as the database is reachable.

With the ingress:

```
echo "$(kubectl -n rdas get ingress rdas-ingress -o
      jsonpath='{.status.loadBalancer.ingress[0].ip}') rdas.local" | sudo tee -a
      /etc/hosts
curl http://rdas.local/api/v1/continents
```

6. External database mode

If MySQL runs outside the cluster, skip `mysql-deployment.yaml` and apply `external-mysql.yaml` instead. It maps the in-cluster name `mysql` to your external host via an `ExternalName` Service, so no app change is needed - only the credentials in `secret.yaml`.

```
kubectl apply -f k8s/external-mysql.yaml
```

7. Configuration reference

Injected from the ConfigMap (non-sensitive) and Secret (credentials):

Variable	Source	Example
SPRING_PROFILES_ACTIVE	ConfigMap	prod (JSON logs + MySQL)
DB_HOST , DB_PORT , DB_NAME	ConfigMap	mysql , 3306 , rdas
RDAS_SOAP_ENDPOINT	ConfigMap	oorsprong CountryInfoService URL
RDAS_HARVEST_CRON	ConfigMap	0 15 2 * * *
RDAS_HARVEST_STALE_AFTER_HOURS	ConfigMap	24
DB_USERNAME , DB_PASSWORD	Secret	rdas / rdas

Apply config changes and restart:

```
kubectl -n rdas apply -f k8s/configmap.yaml
kubectl -n rdas rollout restart deployment/rdas-app
```

8. Scaling, updates, rollback

```
kubectl -n rdas scale deployment/rdas-app --replicas=4           # manual
kubectl -n rdas get hpa rdas-app -w                             # autoscaling
kubectl -n rdas set image deployment/rdas-app rdas-app=reference-data-aggregation-
    service:1.1.0
kubectl -n rdas rollout status deployment/rdas-app              # zero-downtime
    (maxUnavailable 0)
kubectl -n rdas rollout undo deployment/rdas-app                # rollback
```

9. Teardown

```
kubectl delete namespace rdas
```

For runtime issues, see the companion [Kubernetes Troubleshooting Guide](#).